

Advanced Python for Data Science

Week(4): Software Design and Testing

Tuesday, 7th July 2026, 09:30-12:45

Dr. Ghadeer Mobasher

What We'll Cover Today



Software & Clean Code

Why design matters, naming, magic numbers, small functions



Modular Design & Standards

Project structure, PEP 8, Black & Flake8



SOLID Principles

Single Responsibility & Open/Closed



Test-Driven Development

Red-green-refactor, pytest, parametrize testing



Mocking Strategies

Simple mocks, patching, side effects, assertions, class mocks



Documentation

Docstrings and Sphinx

1. FOUNDATIONS



Why Software Design Matters

Small scripts grow into large systems. Without deliberate design, that growth makes code hard to maintain and extend.

Readability

Maintainability

Extensibility

Testability

Collaboration

Poorly Designed Code

```
def process(data):  
    result = []  
    for i in range(len(data)):  
        if data[i] > 10:  
            result.append(data[i] * 2 +  
5)  
    return result
```

What's the issue?

- Unclear purpose → no name reveals intent
- Magic numbers 10, 2, 5 with no meaning
- Manual indexing instead of iteration
- Cannot be unit-tested in isolation

2. CLEAN CODE PRINCIPLES



Meaningful Naming

Clean code prioritises readability. Code should communicate its purpose clearly, so names should say what they do.

Bad

```
def f(x):  
    return x * 2
```

Improved

```
def double_value(value):  
    return value * 2
```



A good name answers: what does this hold, or what does this do? If you need a comment to explain a name, the name isn't good enough yet.

2. CLEAN CODE PRINCIPLES



Avoid Magic Numbers

Unexplained literal values force the reader to guess what they mean and make changes error-prone.

Bad

```
def calculate_price(price):  
    return price * 1.19
```

Improved

```
VAT_RATE = 0.19  
def calculate_price(price):  
    return price * (1 + VAT_RATE)
```



Naming the constant documents its meaning once and lets it be changed in a single place.

2. CLEAN CODE PRINCIPLES



Small Functions, Single Task

Large functions often perform multiple tasks at once, mixing I/O, transformation, and calculation.

Bad → does three things at once

```
def load_and_process(file_path):  
    with open(file_path) as f:  
        data = f.readlines()  
    cleaned = []  
    for line in data:  
        cleaned.append(int(line.strip()))  
    total = sum(cleaned)  
    return total / len(cleaned)
```

Improved → split by responsibility

```
def load_lines(path):  
    with open(path) as file:  
        return file.readlines()  
def clean_numbers(lines):  
    return [int(x.strip()) for x in  
lines]  
def average(values):  
    return sum(values) / len(values)
```

3. MODULAR DESIGN



Organising Code Into Modules

Large systems should be split into modules with a clear separation between source code, tests, and entry point.

```
project/
├── src/
│   ├── loader.py
│   ├── processor.py
│   └── statistics.py
├── tests/
│   ├── test_loader.py
│   └── test_statistics.py
├── requirements.txt
└── main.py
```

loader.py reads raw input

```
def load_numbers(file_path):
    with open(file_path) as f:
        return [int(l) for l in f]
```

statistics.py computes metrics

```
def compute_mean(values):
    return sum(values) / len(values)
```

main.py wires modules together

```
from loader import load_numbers
print(compute_mean(load_numbers(...)))
```

3. MODULAR DESIGN



PEP 8 Coding Standards

PEP 8 is the official style guide for Python code → consistent style makes code predictable to read.

Rule	Example
Functions use snake_case	compute_mean()
Classes use CapWords	DataLoader
Constants use UPPERCASE	MAX_VALUE
Indentation uses four spaces	standard

Not Compliant

```
def CalculateMean(DataList):  
    sum=0  
    for i in DataList:  
        sum=sum+i  
    return sum/len(DataList)
```

Corrected

```
def calculate_mean(data_list):  
    total = 0  
    for value in data_list:  
        total += value  
    return total / len(data_list)
```

Enforcing Style: Black & Flake8

Automated tools remove the guesswork from style compliance → one formats, the other verifies.



Black : the formatter

- Fixes indentation automatically
- Removes inconsistent spacing
- Splits overly long expressions
- Standardises commas & operators

```
pip install black
black --diff script.py
```



Flake8 : the linter

- Detects PEP 8 style violations
- Flags indentation issues
- Catches missing/extra whitespace
- Warns about issues Black won't fix

```
pip install flake8
python -m flake8 file.py
```

4. SOLID PRINCIPLES



Single Responsibility Principle

Each class should have exactly one reason to change → one responsibility.

Bad → two responsibilities in one class

```
class Report:
    def generate(self):
        print("Generating report")
    def save(self, path):
        with open(path, "w") as f:
            f.write("Report")
```

Improved → one class, one job

```
class ReportGenerator:
    def generate(self):
        return "Report content"
class ReportSaver:
    def save(self, content, path):
        with open(path, "w") as f:
            f.write(content)
```

4. SOLID PRINCIPLES



Open/Closed Principle

Software entities should be open for extension but closed for modification.

```
class Discount:
    def apply(self, price):
        raise NotImplementedError
```

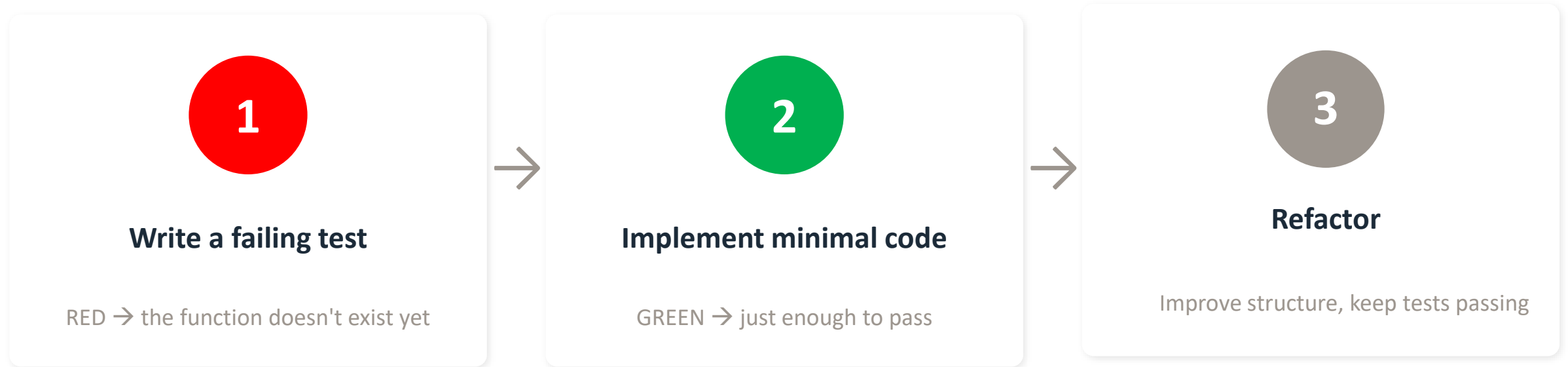
```
class StudentDiscount(Discount):
    STU_RATE = 0.8
    def apply(self, price):
        return price * STU_RATE
class VIPDiscount(Discount):
    VIP_RATE = 0.7
    def apply(self, price):
        return price * VIP_RATE
```



New discount types are added by creating new subclasses → the base Discount class never needs to change.

The Red-Green-Refactor Cycle

In TDD, the developer writes a failing test first, then the minimal code to pass it, then refactors while the test stays green.



pytest automatically discovers and executes test functions

TDD in Practice: square()

A minimal project layout keeps source code and its test side by side.

```
project/  
├─ square.py  
└─ test_square.py
```

Red Stage

```
def test_square():  
    assert square(4) == 16
```

Fails: square() undefined

Green Stage

```
def square(x):  
    return x * x
```

Passes: minimal implementation

Extend & Refactor → grow the test suite as behaviour is confirmed

```
def test_square_positive(): assert square(4) == 16  
def test_square_zero():    assert square(0) == 0  
def test_square_negative(): assert square(-3) == 9
```

Why do we Mock?

Mocking isolates the code under test by replacing external dependencies with controlled, simulated objects.



Simple Mocks

Dummy objects returning predefined values



Patch Mocking

Temporarily replace a target during a test



Side Effects

Simulate exceptions or sequences of values



Assertions on Mocks

Verify how a mock was called



Mocking Classes

Replace entire classes with mock instances

Common target: databases, APIs, file systems → anything external to the unit being tested

Replacing an Object With a Dummy

A simple mock returns a predefined value, standing in for a real dependency such as an API client.

my_module.py

```
def fetch_user(api_client):  
    return api_client.get_user()
```

test_my_module.py

```
from unittest.mock import Mock  
def test_fetch_user():  
    mock_api = Mock()  
    mock_api.get_user.return_value = {"name":  
    "Alice"}  
    assert fetch_user(mock_api) ["name"] ==  
    "Alice"
```

The real API is never called → get_user() is replaced by a controlled stand-in, so the test is fast and deterministic.

Temporarily Replacing a Target

`patch()` swaps a target object, function, or module with a mock for the duration of a test, then restores it.

`my_patch.py`

```
def read_file(path):  
    with open(path, "r") as f:  
        return f.read()
```

`test_patch.py`

```
from unittest.mock import mock_open, patch  
with patch("builtins.open",  
           mock_open(read_data="Hello")):  
    result = read_file("file.txt")
```

`builtins.open` is patched, so `read_file()` reads simulated content → no real file needed on disk.

Simulating Sequences and Exceptions

side_effect lets a mock return different values on successive calls or raise an exception on demand.

```
from unittest.mock import Mock
from my_side_effects import fetch_user
def test_fetch_user_side_effect():
    mock_api = Mock()
    mock_api.get_user.side_effect = [
        {"name": "Alice"}, {"name": "Bob"}
    ]
    result1 = fetch_user(mock_api)    # -> Alice
    result2 = fetch_user(mock_api)    # -> Bob
```

Each call consumes the next item in the list → ideal for testing retry logic or error handling paths.

A simple summary

Strategy	What is replaced?	When to use it?
Simple Mock	A passed object	When the dependency is already a function argument.
Patch	Existing function/module	When the dependency is created internally (e.g. <code>open()</code> , <code>requests.get()</code>).
Side Effect	The behaviour of a mock	When different calls should return different values or raise exceptions.
Assertions	Call history	To verify that a dependency was called correctly.
Class Mock	An entire class	When replacing complex objects like databases or clients.

Docstrings

Docstrings are the standard way to document Python code, read by both humans and tools like Sphinx.

```
def square(x):  
    """  
    Compute the square of a number.  
    Args:  
        x (int | float): The number to be squared.  
    Returns:  
        int | float: The square of the input number.  
    Raises:  
        TypeError: If x is not an int or float.  
    """  
    if not isinstance(x, (int, float)):  
        raise TypeError("input must be numeric")  
    return x * x
```



- Use triple quotes for docstrings
- Describe purpose, args, return value, exceptions
- Follow PEP conventions consistently
- `help(square)` and IDE hover pop-ups display this text automatically

Generating Docs With Sphinx

Sphinx is a Python documentation generator that builds browsable, searchable documentation directly from docstrings.

```
pip install sphinx
```

*Documentation is only useful if it stays current
→ write the docstring the same moment you
write the function.*



Why Sphinx?

- Builds HTML / PDF docs from docstrings
- Keeps documentation in sync with code
- Widely used across the Python ecosystem
- Supports cross-references and search

Team D

15-20 minutes presentation

Team Homelander

15-20 minutes presentation

Pause / Break

See you in 15 minutes 😊!
Please be on time.

Exercises

Solve Problems (1 -5) in "**Week4_Exercise.ipynb**" and you have 45 minutes to finish all the tasks. **Don't forget to split your screen. Read the whole instructions first and answer the reflection questions in your own words.**

Problem 1: Clean Code Refactoring

Objective

Improve readability and maintainability using clean code principles.

Starter Code

Assume that you have `prob1.py` with the following code

```
def f(a,b,c):
    x=a+b
    y=x*c
    return y
```

Tasks

1. rename the function using a meaningful name
2. rename the variables
3. improve indentation and readability

Questions

1. Why are descriptive names important?
2. How does formatting improve maintainability?

Problem 2: Modular System Design

Objective

Learn how to split a program into modules.

Starter Code

```
import statistics

def process(file_path):
```

Please submit your in-class exercises via Moodle.

Key Takeaways

- Design and naming choices compound → small clarity decisions keep systems maintainable.
- PEP 8, Black, and Flake8 make style automatic instead of a matter of opinion.
- SOLID principles keep classes focused and open to extension without rewrites.
- TDD's red-green-refactor cycle builds correctness in before complexity grows.
- Mocking isolates units under test from databases, APIs, and file systems.
- Docstrings and Sphinx turn documentation into a living part of the codebase.

Take-Home Assignment

Complete the following challenge at home. Remember to split your screen and solve the problems. You may always Google, check the syntax, resolve errors, and brainstorm, but **do not generate code or answers**. See the [submission instructions](#).

Mini Challenge: Clean Code, TDD & Mocking

Scenario

You've inherited a `ReportManager` class from a previous developer:

```
class ReportManager:
    def __init__(self, api_client):
        self.api_client = api_client

    def run(self):
        raw_data = self.api_client.get_data()

        total = 0
        for record in raw_data:
            total += record["value"]
        average = total / len(raw_data)

        report_text = f"Report\n-----\nTotal: {total}\nAverage: {average}\n"

        with open("report.txt", "w") as f:
            f.write(report_text)

        return report_text
```

It works, but it breaks almost every principle from this week:

- It does **too many things** in one method (fetch, process, format, save) → violates **Single Responsibility Principle (SRP)**.
- It's **hard to test** because the API call and the file write are buried inside the same method.
- It has **no tests at all**.

Your job: redesign it, then prove it works with tests including a mocked external dependency.

Task 1: Refactor for SRP

Split `ReportManager` into **three separate classes**, each with exactly one responsibility:

- The deadline is 12.07.2026 @ 23:59.
- Starting from this week, you are required to **export the chat history of your conversation with any AI tool you choose to use** and submit it together with your notebook
- See you next time when we dive into **Web Development & API Design**

Any Questions?

See you next time 😊

Dr. Ghadeer Mobasher

ghadeer.mobasher@srh-hochschulen.de

<https://www.linkedin.com/in/ghadeermobasher/>

LGS 6, 2. OG, Arc 213

